

Integrating PQC in OpenSSL via Shallow Providers for Cryptographic Agility

Akif Mehmood¹ and Nicola Tuveri¹

Tampere University, Tampere, Finland {akif.mehmood,nicola.tuveri}@tuni.fi

Abstract. The emergence of Cryptographically Relevant Quantum Computers (CRQCs) threatens traditional cryptographic systems, necessitating a transition to Post-Quantum Cryptography (PQC). OpenSSL 3.0 introduced **Providers**, enabling modular cryptographic integration. This work presents the concept of a *shallow Provider*, facilitating integration of external implementations, to achieve a higher degree of cryptographic agility. **aurora**, which we introduce as an instance of the *shallow Provider* methodology, integrates standardized PQC algorithms in TLS 1.3 for both key establishment and authentication, to support the PQC transition. It enhances cryptographic agility by allowing OpenSSL to dynamically adapt to evolving PQC standards and the rapidly evolving ecosystem of PQC implementations.

Keywords: OpenSSL · Shallow Provider · Post-Quantum Cryptography · Applied Cryptography · Secure Internet Browsing

1 Introduction

Quantum computing was first introduced by Richard Feynman in 1981, theorizing the creation of computers that could harness quantum physics principles. Albeit slowly at first, advances in the field over recent decades pose now a concrete threat to the technology which secures our communications today [9]. While constrained quantum computers have been built, and commercial deployments are available for researchers worldwide, what threatens traditional cryptographic systems, upon which modern Internet browsing relies, is the forecasted emergence of a Cryptographically Relevant Quantum Computer (CRQC), much larger in scale. A CRQC is defined as a quantum computer capable of running algorithms which break or weaken today’s cryptographic systems, such as Rivest-Shamir-Adleman (RSA). While forecasts by experts regarding the emergence of a CRQC within the next ten years are between 19% and 34%, adversaries today *already* have access to the technology for “Harvest Now, Decrypt Later” (HNDL) attacks [31].

To address this threat, research communities around the globe are working on Post-Quantum Cryptography (PQC), to define cryptographic algorithms which run on our current devices, and are believed to be secure even against an attacker with access to a CRQC.

In this paper, pursuing quantum-resistant Transport Layer Security (TLS) connections, we introduce a novel approach for integrating implementations of standardized PQC algorithms into OpenSSL, a cryptographic library widely used across the Internet. We refer to this approach as “*shallow Provider*”, and we will present it in [Section 3](#).

Our research questions (RQs) are:

RQ1. How can a conventional OpenSSL **Provider** be made *shallow*?

RQ2. How does the *shallow Provider* approach facilitate the integration of PQC into OpenSSL?

RQ3. What are the implications of integrating a *shallow Provider* into OpenSSL?

The rest of the paper is organized as follows. [Section 2](#) covers background and related works, and in particular it presents fundamental OpenSSL concepts necessary to describe our approach. [Section 3](#) presents our methodology based on *shallow* loadable modules. [Section 4](#) presents our results applying the *shallow* loadable module approach specifically for the PQC transition and compare them against relevant related works. Finally, in [Section 5](#), we present our conclusions, comparing our contributions against the stated RQs, examining limitations, challenges, and directions for future work.

2 Background

OpenSSL is a commonly used open-source toolkit for cryptographic operations and secure network communication, due to its versatility and robustness. Among other protocols, it provides an open-source implementation of TLS [\[52\]](#) and is primarily used by web servers to secure HTTPS connections. Over its 25-year history, the design and implementation of OpenSSL have been subject to rigorous community scrutiny and continuous security evaluations. The OpenSSL library is written mostly in the *C* programming language and optimized with assembly language to perform essential cryptographic operations and provide a suite of auxiliary functions. Its compatibility spans across a wide array of *Unix-like* operating systems, in addition to *Microsoft Windows* and *OpenVMS*.

Despite its popularity, the complexity of OpenSSL can make its architecture challenging to understand. A closer look at its architecture reveals that OpenSSL’s modular design allows different cryptographic components to be developed, updated, and extended. The architecture [\[45\]](#) of OpenSSL comprises mainly three elements, described below:

2.1 Applications

Conceptually, this represent the set of command-line tools released as part of the project which leverage the underlying `libssl` and `libcrypto` libraries to offer various cryptographic functions and additional features, including parameters and keys handling; test tools for TLS connections; creation and examination of certificates; and more.

2.2 libssl

This is the part of OpenSSL which supports Internet protocols for secure communication such as TLS, Datagram Transport Layer Security (DTLS), and more. `libssl` depends on high-level interfaces of `libcrypto` to handle cryptographic objects and perform cryptographic operations.

2.3 libcrypto

This is the fundamental part of the overall OpenSSL architecture, as it provides access to all the cryptographic primitives, in addition to a rich set of functions to create, manage, and interact with cryptographic objects, such as keys and certificates encoded in various formats. Moreover, `libcrypto` also contains subsystems to enable portability across different platforms and computer architectures, an error stack to consistently handle failures, and more.

Among all these subsystems collectively forming `libcrypto`, we highlight one called *Core*, as it holds a key significance for the work described in this paper: almost any interaction between the `libcrypto` subsystems (and their external callers) and *Providers* is mediated by the *Core* subsystem, which we will briefly discuss below.

As illustrated later in [Figure 1](#), conceptually, *Core* comprises five components named *Global Properties*, *Search Cache*, *Dispatch Table*, *Algorithm Search*, and *Provider Load + Init*. Each of these components is responsible to handle interactions between the rest of `libcrypto` and the inner state of OpenSSL, and to mediate its interactions with *Providers*, which hold the actual implementations for cryptographic operations, key management, encoders and decoders, and access to platform-specific features. In this work, when we use the term *Provider*, we refer to the following definition from the official documentation of OpenSSL [63]: “A *Provider*, in OpenSSL terms, is a unit of code that provides one or more implementations for various operations for diverse algorithms that one might want to perform.” We present more details about *Providers* in the next section.

Furthermore, another important aspect to highlight describing the architecture of `libcrypto` is that the modular design of its many subsystems, as well as access to their functionality within `libcrypto` and externally from other applications and libraries, is made possible by a rich set of Application Programming Interfaces (APIs). While a thorough description of each of these interfaces is out of scope for this paper, here we classify them into two broad categories.

High-level interfaces are typically designed to work across all kinds of algorithms within OpenSSL. For instance, functions provided by the *EVP* API [62] can be used to perform symmetric encryption, digital signatures, key exchange, Hybrid Public Key Encryption (HPKE) [7], and more. Moreover, given an *EVP* object associated with some key, OpenSSL will look up the corresponding implementation based on the key capabilities, abstracting this complexity away from application developers.

Low-level interfaces, conversely, directly expose to application developers the selection of cryptographic operations to be used. For example, an application developer directly calls a specific RSA encryption function rather than letting OpenSSL find the appropriate algorithm based on runtime key capabilities. While these functions used to be popular, since OpenSSL 3.0, they have been deprecated [64]. Legacy applications, still using these deprecated APIs, cannot fully leverage the OpenSSL 3.0+ **Provider** architecture. Therefore, in this work, we only consider OpenSSL-based applications which exclusively use high-level interfaces.

Built-in OpenSSL Providers Internally, `libcrypto`, delegates cryptographic operations to the providers available at runtime. Natively, OpenSSL 3.0+ comes with five built-in **Providers**, each supporting a different set of algorithms, as discussed below.

Default Provider includes all of the most widely used algorithms such as Advanced Encryption Standard (AES) [1] and RSA [30]. It is integrated within `libcrypto`. Explicitly loading the *Default Provider* is not required in general as it is automatically loaded if no other **Provider** is specified.

Legacy Provider contains deprecated algorithms often considered insecure, such as MD5 [53] and RC4 [47]. It needs to be loaded explicitly, either through OpenSSL configuration file or programmatically, if such legacy algorithms are required.

FIPS Provider is routinely validated via an accredited testing laboratory for conformance with FIPS 140 [33], and therefore contains only the subset of algorithms which meet the validation criteria among those included in the *Default Provider*. Installation and configuration of this **Provider** must follow the associated Security Policy document for users to leverage the Federal Information Processing Standards (FIPS) Approved mode of operation.

Base Provider includes algorithms that are used for encoding and decoding of keys and cryptographic objects. These encoders and decoders are usually excluded from the FIPS validation process, and this **Provider** exists mostly to support the **FIPS Provider**, which contains only cryptographic implementations.

Null Provider is a built-in **Provider** for `libcrypto` like every other. It does not contain any algorithm at all. It exists mainly to prevent automatic loading of the *Default Provider*, when needed.

Additionally, external **Providers** can also be loaded at runtime to serve different purposes. Although external **Providers** could be embedded within applications themselves, it is arguably more common to deploy external **Providers** in the form of loadable modules. For the rest of this manuscript we will focus exclusively on loadable external **Provider** modules, as these afford the flexibility goals that will be discussed later.

2.4 OpenSSL Provider Internals

As illustrated at the top of [Figure 2](#), an OpenSSL **Provider** comprises primarily three components, i.e., *Provider Init function*, *Provider Query Implementation*, and *Algorithm Implementation*. Each of these components has a distinct role in enabling cryptographic operations and supporting the extensibility of OpenSSL. Below we summarize these components and their purposes within a **Provider**.

Provider Init function is the entrypoint of any **Provider**. This is an initialization function that interacts with the *Core* subsystem of OpenSSL’s `libcrypto`. The main responsibility of this function is to make sure that other components within a **Provider** are successfully loaded and then report back to the OpenSSL *Core*.

Provider Query Implementation is the component of a **Provider** responsible for answering queries received from the *Core* subsystem, as illustrated in [Figure 1](#). These queries mostly happen during the “Algorithm Fetching” step, and this component interacts with “**Provider Query Implementation**” to list available implementations which reside within the **Provider**.

Provider Algorithm Implementations. A **Provider** generally contains multiple algorithm implementations. This block is an abstract representation of all the actual cryptographic implementations available within the **Provider**.

2.5 NIST PQC Project

The US National Institute of Standards and Technology (NIST), has been at the forefront of all the recent cryptographic transitions, and has been actively spearheading the standardization of PQC since 2016 [\[38, 32\]](#).

The NIST PQC project [\[42\]](#) is an international collaborative effort to standardize cryptographic algorithms secure against quantum computers, in a multi-round process. In July 2022, at the end of the third round of the process, NIST selected algorithms as finalists for standardization [\[59\]](#). These selections were: CRYSTALS-Kyber [\[3\]](#) as the primary Key Encapsulation Mechanism (KEM) for key establishment, and CRYSTALS-Dilithium [\[4\]](#), FALCON [\[17\]](#), and SPHINCS+ [\[2\]](#) as digital signature schemes for authentication. After the publication of Initial Public Drafts (IPDs) [\[37\]](#) and a year to collect feedback, NIST published as FIPS-approved standards in August 2024 [\[36\]](#):

- FIPS 203** [\[35\]](#) Module-Lattice-Based Key Encapsulation Mechanism (ML-KEM) based on CRYSTALS-Kyber, allows to establish a shared secret key secure against quantum attackers.
- FIPS 204** [\[34\]](#) Module-Lattice-Based Digital Signature Algorithm (ML-DSA) based on CRYSTALS-Dilithium, provides message authentication.

FIPS 205 [39] Stateless Hash-based Digital Signature Algorithm (SLH-DSA) provides authentication and data integrity through the SPHINCS+ construction.

FIPS 206 Although it has been announced [36], as of August 2025, NIST still has to publish the FIPS 206 IPD based on FALCON, dubbed FFT (fast-Fourier transform) over NTRU-Lattice-Based Digital Signature Algorithm (FN-DSA).

Excluding SLH-DSA, all the standards above build their security over the computational hardness of problems that involve structured lattices. Additional non-lattice-based KEMs remained under consideration in the fourth round of the standardization process, but no other signature scheme proposal remained. In order to diversify its portfolio of PQC signatures, upon the end of the third round of the process, NIST also started a parallel process [41] to select additional PQC signature schemes, which is currently in its second round, featuring 14 candidates across 6 categories of computationally hard problems (including one lattice-based candidate related to FALCON which has been retained under consideration due to its large performance advantage over the currently selected lattice-based schemes) [57]. In March 2025, NIST announced [58] the selection of Hamming Quasi-Cyclic (HQC) [29] as an additional KEM to standardize, joining ML-KEM as backup.

2.6 Bridging PQC Algorithms and Protocols

As discussed in the previous section, NIST and other similar national Standards Defining Organizations (SDOs) are responsible for selecting and ratifying cryptographic algorithms. The task of integrating these algorithms into the protocols that power the Internet is carried out by engineering and research communities such as Internet Engineering Task Force (IETF) and Internet Research Task Force (IRTF). The IETF acts as the main SDO for the Internet, focused on improving it by producing technical documents that guide its development and management.

The QUBIP project is mainly interested in the Working Groups (WGs) and research groups that address the PQC transition. Such groups inside IETF and IRTF are primarily PQUIP, CFRG, TLS, and LAMPS, as detailed below.

- Post-Quantum Use In Protocols (PQUIP) is an IETF WG that provides a standing venue to discuss PQC transition issues and, where no dedicated group exists, PQC-related concerns in IETF protocols. Notably, PQUIP is maintaining a living document that tracks the state of PQC adoption and migration of protocols and applications [22]. It also has a number of Internet-Drafts (I-Ds)—and recently published its first Request for Comments (RFC)—defining terminology, exploring design spaces, and providing guidance about different aspects of the PQC transition [e.g., 16, 10, 5, and more].

- Crypto Forum Research Group (CFRG), an IRTF group, bridges theory and practice by promoting new cryptographic techniques for the Internet community, and serves as a forum for related discussions. In PQC-related work, the CFRG formed a design team focused on Post-Quantum/Traditional (PQ/T) Hybrid KEMs. This team has produced an I-D investigating the design space [13], followed by another defining concrete instantiations with specific schemes and parameter sets [12], to fulfill the goals and generic constructions outlined in the first.
- The Limited Additional Mechanisms for PKIX and SMIME (LAMPS) WG maintains protocols, formats, and extensions concerning certificates and authentication, and play a pivotal role in improving Public Key Infrastructures (PKIs), particularly in relation to S/MIME [56], CMS [19], and Public Key Infrastructure (X.509) (PKIX) [11]. Within LAMPS, the PQC-related work is progressing extensively. In this regard, many I-Ds are relevant, including:
 - [27] deals with the adoption of ML-DSA in PKIX.
 - [46] addresses the use of ML-DSA composite signatures (i.e., one form of PQ/T Hybrid authentication) in PKIX.
 - [8] deals with the adoption of SLH-DSA in PKIX.

Furthermore, since IETF 115, within the co-located IETF Hackathon, a group [21] has been actively working on designs and experiments for the interoperable deployment of PQC in certificates. To date, the major goals of these endeavors have been:

- Integrating PQC algorithms support into existing X.509 structures and ensuring support across a diverse set of implementations, while following the updates around the various standardization efforts.
- Establishing a repository for automatic interoperability testing across different implementations.
- Creating a comprehensive compatibility matrix to document the collected results.
- Provide feedback about practical usage to the relevant IETF WGs.
- TLS is an IETF WG that works on the homonymous protocol, ensuring secure Internet communication, providing confidentiality, integrity, and authentication. Most of the PQC transition efforts mainly target the latest version of the protocol, TLS 1.3 [52].

Recent PQC work addresses the design of PQ/T Hybrid Key Exchange [60] for TLS 1.3, to simultaneously use multiple key exchange algorithms and combine their results for better security assurances. A separate I-D adopted by the TLS WG [24] instantiates this generic design for specific combinations of traditional and PQC schemes:

- X25519MLKEM768 combines X25519 [25] with ML-KEM-768. It has been deployed at scale at Google and Cloudflare, and is currently supported in official versions of Google Chrome and Mozilla Firefox.
- SecP256r1MLKEM768 combines NIST P-256 [6, 49] with ML-KEM-768. It derives TLS session keys with the same equivalent level of security of the hybrid above, but adopting a FIPS-approved traditional scheme.

- **SecP384r1MLKEM1024** combines NIST P-384 [6, 49] with ML-KEM-1024. It derives TLS session keys with a higher level of security than the hybrids above, adopting a FIPS-approved traditional scheme.
- the I-D also obsoletes the experimental entries in the TLS Supported Groups registry [20] for former draft groups [66, 23] which had previously seen some widespread deployment, and have since been replaced by the corresponding new hybrids described above.

Moreover, the WG is also working on I-Ds defining the use of PQC algorithms for authentication of the TLS handshake, such as [18] for ML-DSA, [51] for SLH-DSA, and [50] for PQ/T Composite ML-DSA.

2.7 Cryptographic agility

Section 2.5 already demonstrated how the set of cryptographic schemes available to counter the threat of CRQCs is in constant flux, while Section 2.6 illustrated how the process by which protocols and applications identify and then adopt the standardized schemes which best suit their use cases and constraints is complex and varied. Moreover, it requires regular revision as cryptographic recommendations and requirements evolve at both national and international levels.

Historically, these cryptographic transitions have been expensive, spanning decades, creating interoperability challenges, and disrupting operations [14, Sec. 2]. Current technological trends also suggest that the ongoing PQC transition will not be the last.

These considerations motivate the adoption of new cross-disciplinary strategies and mechanisms to facilitate smoother transitions for any application of cryptography, leading to the recognition of “cryptographic agility” as a foundational design goal for modern systems. In this context, across the many definitions available in the literature (e.g., [26, p. 3] and [14, App. B]), we adopt the following:

Cryptographic (crypto) agility describes the capabilities needed to replace and adapt cryptographic algorithms for protocols, applications, software, hardware, firmware, and infrastructures while preserving security and ongoing operations. — (NIST CSWP 39 (2PD) [14]).

Within our implementation of **aurora**¹, discussed in Section 4, we *further extend the notion of crypto agility*. Beyond enabling the replacement of cryptographic schemes, we recognize that applications and users often require the flexibility to select among alternative implementations of the same scheme. Among many others, such flexibility supports diverse goals: e.g., specialized implementations optimized for performance in specific deployments, formally verified code offering stronger security assurances, access to dedicated hardware in constrained environments, or rapid replacement in response to discovered implementation defects.

¹ <https://github.com/QUBIP/aurora>

All these concerns are especially critical at the cryptographic library layer, which must provide secure communications across a wide variety of applications, users, and operating environments.

2.8 Related Work

To contextualize our work, we have analyzed related research and solutions. Among these the most prominent is the Open Quantum Safe (OQS) project [61]. The OQS project is an open-source initiative aimed at facilitating the transition to PQC. It offers the `liboqs` [43] library, which collects implementations of many algorithms from the ongoing NIST PQC project [42]. Additionally, the OQS project published the `OQS Provider` [44], which integrates `liboqs` algorithms into OpenSSL using its `Provider` architecture.

Together, they provide access to various KEM and signature algorithms which have already been standardized, such as ML-KEM and ML-DSA, alongside implementations of other submissions, and various PQ/T Hybrid schemes. Due to this breadth of supported algorithms and the maturity level of the project, `liboqs`, and `OQS Provider`, have been deployed on large scale both in experimental and production-ready contexts. Due to this very reason, we selected `OQS Provider` as our functional compatibility target for the QUBIP project, and `aurora` will contain only the NIST standardized PQC implementations.

3 Methodology

According to the Linux Kernel Module Programming Guide [55]: “Modules are pieces of code that can be loaded and unloaded into the kernel upon demand.” Keeping a similar understanding of loadable modules, the value of plug-and-play or loadable modules in software components that allow new cryptographic implementations into existing systems has been highlighted in recent research [65], and is the foundation for the OpenSSL `Provider` architecture. Loadable modules play a critical role in enabling new security protocols and standards to be promptly included, which is essential for keeping cryptographic systems up-to-date with the constantly evolving cryptography landscape. As discussed in [Section 2.3](#), OpenSSL `Providers` are loadable modules, which can be loaded and unloaded as needed, and which provide access to specific cryptographic implementations to perform the supported cryptographic operations.

In this section, we will explain how OpenSSL `Providers`—and loadable modules more generally—contribute to cryptographic agility.

We note that, in the approach we present here, no particular restriction is imposed on the nature of the external algorithm implementations, while in the next section we will explore in particular how applying this approach specifically for the PQC transition achieves the goals we set for our work.

3.1 The Approach: *shallow* Provider

We introduce the concept of “*shallow Provider*”, a loadable module which is capable of integrating external algorithm implementations and make them interoperate seamlessly with OpenSSL and the applications which leverage the library for their security functions. The term *shallow* refers to the fact that the module—unlike conventional OpenSSL *Providers*—rather than embedding cryptographic implementations within itself is dynamically linked against external libraries providing the actual implementations. In our framework, this is facilitated through an interoperability layer of “*Adapters*” as shown in [Figure 1](#).

Adapters act as a channel through which OpenSSL can communicate with external algorithm implementations. This provides a separation between the external implementations and the internal details of OpenSSL-specific requirements.

This separation serves a dual purpose. On one hand, this decouples OpenSSL from the external implementations in a way that neither OpenSSL nor the external implementations need to know about the details of each other; moreover, in the context of the PQC transition, it decouples the fast-paced development cycles of the rapidly evolving ecosystem of PQC implementations from the generally more stable development iterations of OpenSSL.

Secondly, this separation enhances the reusability of the scaffolding code around *Adapters* which is required to interact with the OpenSSL *Provider* API. By design, this scaffolding is in large part identical for different implementations of the same algorithm, and features considerable overlaps across different cryptographic operations.

This approach is built to integrate harmoniously with OpenSSL and, through modules built according to this paradigm, applications leveraging high-level OpenSSL API are capable of adopting new algorithms without necessitating core architectural changes, achieving cryptographic agility.

Furthermore, as the external implementations can be easily updated independently of OpenSSL or the *shallow Provider*, and as “*Adapters*” facilitate swapping among alternative implementations for a given cryptographic scheme, we can achieve an even higher degree of cryptographic agility.

3.2 Components of the *shallow* Provider Framework

As illustrated in [Figure 1](#), the *shallow Provider* framework consists of four components, i.e., User Application, OpenSSL’s `libcrypto`, the *shallow Provider* itself, and *external implementations*. We defer to the caption of [Figure 1](#) for a summary of the interactions between the first two components and the *Provider* itself. Here we will focus on discussing the last two components.

shallow Provider. The differences between a conventional OpenSSL *Provider* and a *shallow Provider* are highlighted in the top and middle parts of [Figure 2](#).

Firstly, we replaced the “Algorithm Implementations” component with *Adapters*, which seamlessly bridge external algorithm implementations with OpenSSL to

perform secure cryptographic operations. In general, each externally supported implementation will require its own Adapter, irrespective of how many different algorithms each implementation provides. *Adapters* are responsible for initialization of the cryptographic implementation they support, and interact with the “Provider Query Interface” component within a *shallow Provider*.

The “Provider Query Interface” is mutated in this framework as its state also maintains knowledge of all the available *Adapters* in a *shallow Provider*, in addition to answering the queries received from the OpenSSL’s `libcrypto`.

External Implementations. The *shallow Provider* methodology facilitates the integration of external implementations through the use of “Adapters” as mentioned before. An external implementation acts as a backend implementation of any algorithm for a *shallow Provider*. A *shallow Provider* may support any number of external implementations based on the needs of particular use cases. For instance, an instance of a *shallow Provider* could be tailored to only support formally verified software implementations for a given algorithm, while

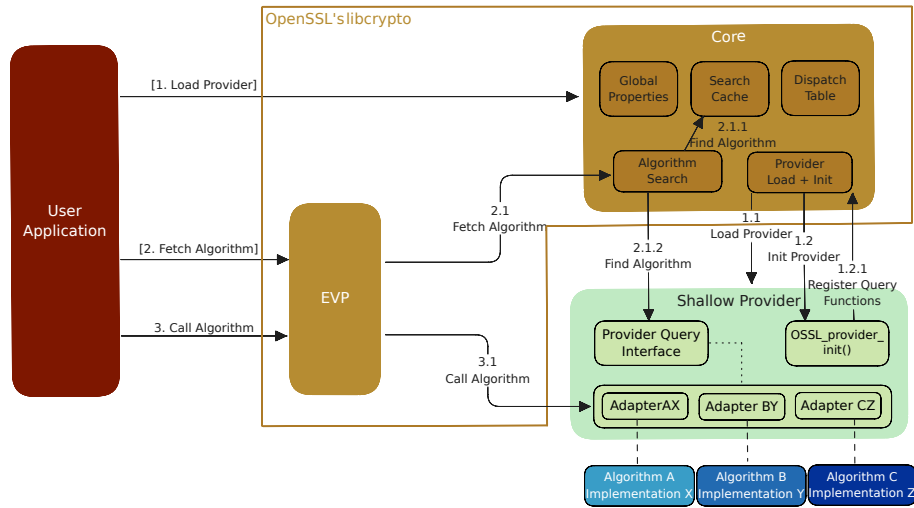


Fig. 1: This diagram illustrates our *shallow Provider* framework. It is closely inspired by the diagram in [OpenSSL 3.0.0 Design Document \[45\]](#). A User Application interacts with OpenSSL via the high-level APIs of `libcrypto` (e.g., `EVP`). The *Core* subsystem of `libcrypto` loads Providers, and fetches algorithms from any loaded Provider to fulfill application requests. Upon loading, a *shallow Provider* initializes its components and responds back to the requests of *Core*. As part of its internal state management *Core* deals with each “Provider Query interface” to discover the functionality supported by the Provider. In turn, our *shallow Provider*, gathers the required information from its embedded “Adapters”, which abstract the interactions with different external backend implementations.

another instance could be configured to rely on a dedicated hardware implementation for the same algorithm. This is achieved by selecting a different “Adapter” supporting the same algorithm, depending on the specific requirements.

In general, the greatest hurdle a *shallow Provider* may face is the fact that different external implementations could have very different interfaces. This can go beyond just different function names: e.g., different implementations of a symmetric cipher may follow different memory management practices for storing keys, or different implementations of an Authenticated Encryption with Associated Data (AEAD) cipher may have different conventions for how to pass in the associated data. *Adapters* take these differences into account and provide compatibility between the expectations of OpenSSL APIs and the specific requirements and behavior of the corresponding external implementation.

4 Results

While in the previous section, supported by [Figure 1](#), we presented the framework generically, here we focus on a comparative analysis between a generic *shallow Provider* and one specific instance of the *shallow Provider* framework, tailored for the PQC transition: **aurora**. We provide [Figure 2](#), as a visual aid to support this analysis. Note that we already discussed the top and middle parts of this diagram in the previous section, when describing the differences between a conventional OpenSSL *Provider* and a *shallow* one, so for the discussion of this section we focus on a comparison of the middle and bottom parts of the diagram, representing, respectively, a generic *shallow Provider* and **aurora**.

First we notice that the representations of their internals are mostly identical, and differ exclusively in the “Adapters” layer. Similarly, the situation with PQC implementations is highly regular, in comparison to traditional cryptographic algorithms. This is because most PQC algorithm implementations conform to a standard NIST API [40], designed as a part of the PQC standardization project [42]. This also applies also to candidate algorithms excluded from the NIST process but under consideration by other SDOs—such as International Organization for Standardization (ISO) and European Union Agency for Cybersecurity (ENISA). Therefore, we tailored **aurora** for the PQC transition adopting the NIST PQC API as the common denominator for all the external implementations we support. This is not a limitation: PQC algorithms for KEMs or digital signature schemes are expected to conform to the same baseline NIST PQC API—outlined in the original 2016 call for submissions—regardless of different recommendations from various SDOs,

We illustrated this in [Figure 2](#) by a dotted line separating the “Adapters” from the external implementations.

Furthermore, while depicting a generic *shallow Provider* in the middle of [Figure 2](#) we chose to clearly draw boundaries between the different Adapters, the illustration of **aurora** at the bottom assimilates multiple adapters in a single abstract “PQ Adapters” block. We made this visualization choice to underline the fact that, once we chose the NIST PQC API as our interoperability layer for

external implementations, the different Adapters started sharing more and more characteristics, leading to increased reusability in our codebase. Fundamentally the unique differences among Adapters within **aurora** can be summarized as follows:

- different back-end implementations require different (dynamic or static) linking information, and each Adapter embeds this information when it is built;
- different back-end implementations have different initialization requirements, which each Adapter needs to uniquely address;
- each adapter needs to list exactly which subset of cryptographic operations it exposes to OpenSSL from the corresponding back-end implementation.

As stated, **aurora** is designed to support arbitrary external implementations, as long as they are compliant with the NIST PQC API. Currently, for **aurora** we instantiated three “Adapters” backed by different libraries that also feature PQC primitives.

- The **libcrux** adapter engages an ML-KEM implementation from the *libcrux* library [15], a formally verified, high-performance cryptographic library written in Rust that unifies verified code across Rust, C, and assembly using the **hax** toolchain.
We hybridize ML-KEM with *libcrux* implementations for X25519 and P-256 to enable support for TLS 1.3 PQ/T Hybrid key exchange, according to [24], in **aurora**.
- The **pqclean** adapter leverages the ML-DSA implementation from *PQClean* [48], a project that aims to collect clean, portable, and thoroughly tested standalone C implementations of NIST-standardized PQC schemes, designed for easy integration and high code quality.
This adapter enables both pure ML-DSA and *composite ML-DSA* [50, 46], providing support for Post-Quantum (PQ) or PQ/T Hybrid authentication.
- The **rustcrypto** adapter embeds an SLH-DSA implementation [54] from *RustCrypto*, a project which publishes and maintains independently versioned crates containing pure-Rust traits and implementations for many different cryptographic schemes.

We specifically mention these three alternatives to suggest the breadth of freedom in picking the implementation that best suits a given use case, or to quickly respond to a newly discovered defect in one implementation of an algorithm for which alternatives are available, or many other scenarios which are enabled by the extra layer of cryptographic agility provided by our framework. While the back-end implementation can be arbitrarily swapped as needed via configuration files specific for a deployment, existing applications leveraging OpenSSL benefit from them without any alteration.

Comparison against related work. In [Section 2.8](#) we discussed the **OQS Provider**, as it also applies the **OpenSSL Provider** paradigm to address the PQ transition.

Here we conclude this section by comparing our work against the OQS Provider to highlight similarities and differences.

First, we remark that, by design, we decided early on to pick OQS Provider as our functional compatibility target, i.e., applications leveraging OpenSSL are not affected by swapping the OQS Provider for *aurora*, as long as they only require support for standardized schemes. This allowed partners in the QUBIP consortium to carry on their development and tests in parallel with the design and implementation work for our Provider. Moreover, this facilitated our own

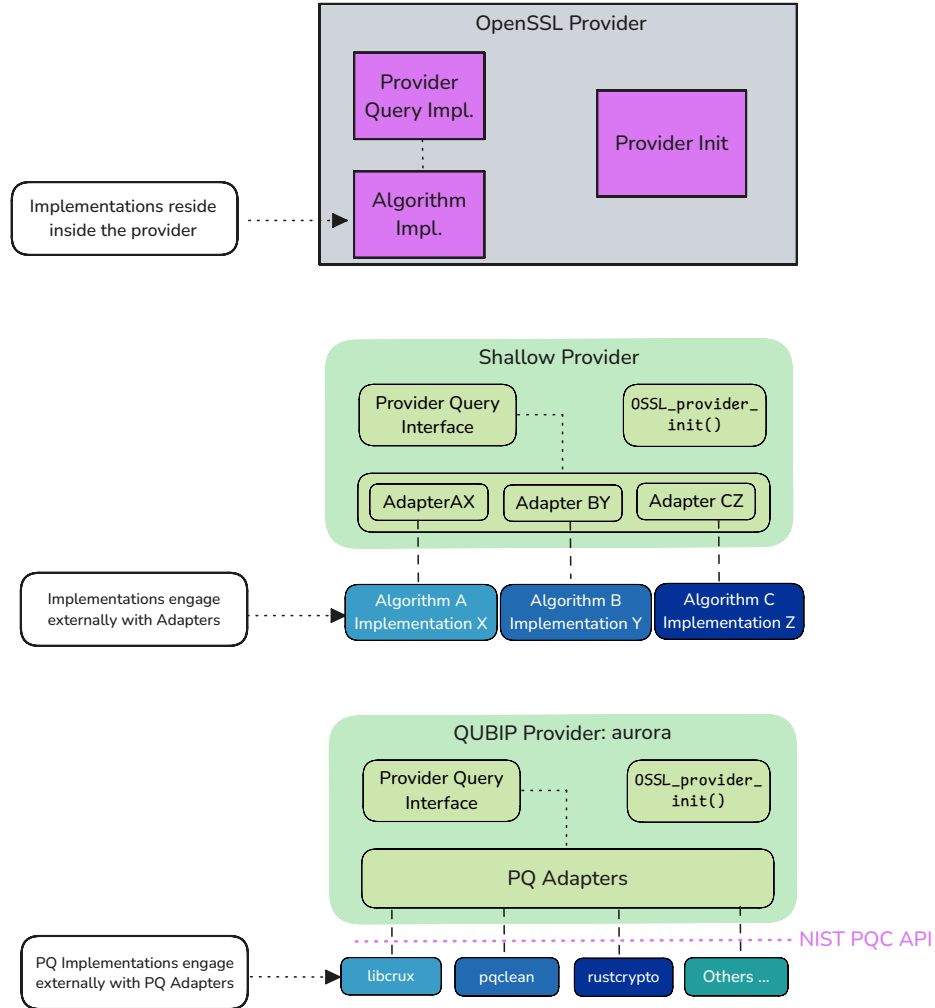


Fig. 2: A visual comparative analysis of a conventional OpenSSL Provider (top), a generic *shallow* Provider (middle), and *aurora* (bottom) as a particular instance of the latter tailored for the PQC transition.

internal development and testing, by providing a target for interoperability testing which was independently kept up to date with the evolving PQC standards.

The main difference between our **Provider** and the **OQS Provider** is that we selected the *shallow Provider* approach aiming for a higher degree of cryptographic agility. The **OQS Provider** relies on **liboqs** for the implementations of the cryptographic operations it supports, and statically links them within itself at build time. The immediate consequence of this fact is that the release schedule of the **OQS Provider** and of **liboqs** are intimately intertwined. Moreover, if practitioners need to replace the implementations of any of the schemes supported by the **OQS Provider**, the path of least resistance goes through maintaining a fork of **liboqs** replacing the corresponding implementations alongside the changes required in their fork of **OQS Provider**. For clarity, we reiterate that modifying the **OQS Provider** to support different implementations than **liboqs** is not unfeasible, but is objectively challenging because the design has not been originally made to accommodate such changes. On the contrary, the *shallow Provider* approach adopted by **aurora**, and its overall design, empower experimenters to plug in their choice of external PQC implementations reusing as much infrastructure as possible and limiting the required changes to small alterations of the existing **Adapters**. This ultimately leads to a lower barrier for adopting arbitrary choices of external implementations—as long as they are compliant with the NIST PQC API—eliminating this limitation.

In the next section, exploring the limits of our work, we will also address the difference in amount of supported algorithms between the two **Providers**.

5 Conclusions and future work

We presented how our approach based on *shallow Providers* supports the PQC transition and demonstrated the feasibility of integrating *external* PQC implementations into OpenSSL-based applications seamlessly. By adopting the *shallow Provider* approach, **aurora** also unlocks a higher degree of cryptographic agility.

In addition to supporting ML-KEM PQ/T Hybrids with X25519 and P-256 for PQ key exchange—backed by *libcrux*—our **Provider** also supports PQ authentication. Specifically, we support ML-DSA leveraging the *PQClean* project and SLH-DSA via *RustCrypto*, thereby extending the coverage to both PQ key exchange and authentication as described in [Section 4](#), with full support for PQ/T Hybrid modes. Moreover, within the QUBIP project, one of the partners leveraged our methodology to derive one more “Adapter” from our initial set, tailoring it to leverage novel hardware-based implementations developed as part of the project. One more “Adapter” currently under development aims to dynamically link to **liboqs** to provide a wider portfolio of experimental algorithms for research. These various “Adapters” demonstrate how we achieved the original goal of supporting diverse backend implementations, pursuing an extended notion of cryptographic agility.

Additionally, we discussed how we achieved parity with `OQS Provider` as the designated functional compatibility target, for both PQC KEM and authentication. In this regard, it is important to note that, pursuing parity in terms of functional compatibility for KEM and authentication, the QUBIP project has focused on covering only the PQC standards selected for deployment, rather than covering all the (often experimental) algorithms covered by `liboqs`.

An open issue we are still addressing concerns a limitation of the current version of `aurora`: “Adapters” are statically linked to external implementations at build time, rather than dynamically at runtime. Although the current design allows dynamic linking, Rust’s tooling defaults to static linkage. We are working on a solution that decouples this behavior from the details of the external implementation, allowing dynamic or static linking to be chosen as needed, with ease.

A drawback of our methodology is the additional computational and memory overheads introduced by the “Adapters” layer. We have monitored its impact throughout our work and, by leveraging Rust’s features and zero-cost abstractions, have been able to mitigate it effectively. Looking ahead, we plan to further reduce this overhead by optimizing the scaffolding around “Adapters”. Our future work will also focus on further exploiting Rust’s capabilities to enable the automatic generation of “Adapters” from declarative descriptions, maximizing code reuse and lowering the surface for programmer errors in the common scaffolding.

As a final note, we highlight that, as part of this work, we also built a foundation for any developer to write their own `OpenSSL Providers` using the Rust language, a systems programming language focused on safety, concurrency, and performance. We named this foundation `openssl-provider-forge`²—a Rust crate which contains Foreign Function Interface (FFI) bindings for `OpenSSL 3.2+`, specifically for its `Core` and `Provider` API. This foundational crate contains commonly required functions and abstractions essential for authoring an `OpenSSL Provider`, lowering the barrier of entry by reducing the burden of setting up common abstractions and patterns, while promoting the reuse of well-tested boilerplate code to avoid subtle pitfalls when interfacing with `OpenSSL` APIs. We hope in the future this will enable more researchers to apply our methodology beyond the PQC scope of the QUBIP project.

Acknowledgments

This work has been developed within the QUBIP project (<https://www.qubip.eu>), funded by the European Union under the Horizon Europe framework programme [grant agreement no. 101119746]. The authors acknowledge that parts of this work are derived from research presented in a master’s thesis completed at Tampere University [28].

² <https://github.com/QUBIP/openssl-provider-forge-rs/>

References

1. Advanced Encryption Standard (AES). FIPS PUB 197, Updated on 2023-05-09. NIST (2001). <https://doi.org/10.6028/NIST.FIPS.197-upd1>
2. Aumasson, J.-P., Bernstein, D.J., Beullens, W., Dobraunig, C., Eichlseder, M., Fluhrer, S., Gazdag, S.-L., Hülsing, A., Kampanakis, P., Kölbl, S., Lange, T., Lauridsen, M.M., Mendel, F., Niederhagen, R., Rechberger, C., Rijneveld, J., Schwabe, P., Westerbaan, B.: SPHINCS+. Tech. rep., (2022). <https://sphincs.org/data/sphincs+-r3.1-specification.pdf>
3. Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber. Tech. rep., (2021). <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf>
4. Bai, S., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Dilithium. Tech. rep., (2021). <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf>
5. Banerjee, A., Reddy, K. T., Schoiniakakis, D., Hollebeek, T., Ounsworth, M.: Post-Quantum Cryptography for Engineers. I-D draft-ietf-pquip-pqc-engineers-13, IETF (2025). <https://datatracker.ietf.org/doc/draft-ietf-pquip-pqc-engineers/13/>
6. Barker, E., Chen, L., Roginsky, A., Vassilev, A., Davis, R.: Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography. NIST SP 800-56A Rev. 3, NIST (2018). <https://doi.org/10.6028/nist.sp.800-56ar3>
7. Barnes, R., Bhargavan, K., Lipp, B., Wood, C.A.: Hybrid Public Key Encryption. RFC 9180, IETF RFC Editor (2022). <https://doi.org/10.17487/RFC9180>
8. Bashiri, K., Fluhrer, S., Gazdag, S.-L., Geest, D.V., Kousidis, S.: Internet X.509 Public Key Infrastructure: Algorithm Identifiers for SLH-DSA. I-D draft-ietf-lamps-x509-slhdsa-09, IETF (2025). <https://datatracker.ietf.org/doc/draft-ietf-lamps-x509-slhdsa/09/>
9. Bernstein, D.J., Lange, T.: Post-quantum cryptography. *Nature* **549**(7671), 188–194 (2017). <https://doi.org/10.1038/NATURE23461>
10. Bindel, N., Hale, B., Connolly, D., D, F.: Hybrid signature spectrums. I-D draft-ietf-pquip-hybrid-signature-spectrums-07, IETF (2025). <https://datatracker.ietf.org/doc/draft-ietf-pquip-hybrid-signature-spectrums/07/>
11. Boeyen, S., Santesson, S., Polk, T., Housley, R., Farrell, S., Cooper, D.: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280, IETF RFC Editor (2008). <https://doi.org/10.17487/RFC5280>
12. Connolly, D., Barnes, R.: Concrete Hybrid PQ/T Key Encapsulation Mechanisms. I-D draft-irtf-cfrg-concrete-hybrid-kems-00, IETF (2025). <https://datatracker.ietf.org/doc/draft-irtf-cfrg-concrete-hybrid-kems/00/>
13. Connolly, D., Barnes, R., Grubbs, P.: Hybrid PQ/T Key Encapsulation Mechanisms. I-D draft-irtf-cfrg-hybrid-kems-05, IETF (2025). <https://datatracker.ietf.org/doc/draft-irtf-cfrg-hybrid-kems/05/>
14. Considerations for Achieving Cryptographic Agility: Strategies and Practices. NIST Cybersecurity White Paper (CSWP) 39 (2nd Public Draft), NIST (2025). <https://doi.org/10.6028/NIST.CSWP.39.2pd>
15. Cryspen, libcrux – The Formally Verified Crypto Library, (2023). <https://cryspen.com/libcrux-library/> (visited on 08/19/2025)
16. D, F., P, M., Hale, B.: Terminology for Post-Quantum Traditional Hybrid Schemes. RFC 9794, IETF RFC Editor (2025). <https://doi.org/10.17487/RFC9794>
17. Fouque, P.-A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z.: Falcon: Fast-Fourier Lattice-based

- Compact Signatures over NTRU. Tech. rep., (2020). <https://falcon-sign.info/falcon.pdf>
18. Hollebeek, T., Schmieg, S., Westerbaan, B.: Use of ML-DSA in TLS 1.3. I-D draft-ietf-tls-mldsa-00, IETF (2025). <https://datatracker.ietf.org/doc/draft-ietf-tls-ml-dsa/00/>
 19. Housley, R.: Cryptographic Message Syntax (CMS). RFC 5652, pp. 1–56. RFC Editor (2009). <https://doi.org/10.17487/RFC5652>. <https://datatracker.ietf.org/doc/rfc5652/>
 20. IANA, IANA Registry: TLS Supported Groups, (2018). <https://www.iana.org/assignments/tls-parameters/tls-parameters.xhtml#tls-parameters-8> (visited on 07/29/2025)
 21. IETF Hackathon, PQC Certificates, (2022). <https://github.com/IETF-Hackathon/pqc-certificates>
 22. IETF PQUIP, State of Protocols and PQC, (2024). <https://github.com/ietf-wg-pquip/state-of-protocols-and-pqc> (visited on 08/23/2025)
 23. Kwiatkowski, K., Kampanakis, P.: Post-quantum hybrid ECDHE-Kyber Key Agreement for TLSv1.3. I-D draft-kwiatkowski-tls-ecdhe-kyber-01, IETF (2023). <https://datatracker.ietf.org/doc/draft-kwiatkowski-tls-ecdhe-kyber/01/>
 24. Kwiatkowski, K., Kampanakis, P., Westerbaan, B., Stebila, D.: Post-quantum hybrid ECDHE-MLKEM Key Agreement for TLSv1.3. I-D draft-ietf-tls-ecdhe-mlkem-00, IETF (2025). <https://datatracker.ietf.org/doc/draft-ietf-tls-ecdhe-mlkem/00/>
 25. Langley, A., Hamburg, M., Turner, S.: Elliptic Curves for Security. RFC 7748, pp. 1–22. RFC Editor (2016). <https://doi.org/10.17487/RFC7748>. <https://datatracker.ietf.org/doc/rfc7748/>
 26. Macaulay, T., Henderson, R.: Cryptographic Agility In Practice: emerging use-cases. Use Cases Whitepaper, InfoSec Global (2019). https://web.archive.org/web/20240918080004/https://cdn.prod.website-files.com/612fec6a451c71c9308f4b69/614b712e53ce8f7fad0c3c4a_ISG_AgilityUseCases_Whitepaper-FINAL.pdf (visited on 09/18/2024)
 27. Massimo, J., Kampanakis, P., Turner, S., Westerbaan, B.: Internet X.509 Public Key Infrastructure - Algorithm Identifiers for the Module-Lattice-Based Digital Signature Algorithm (ML-DSA). I-D draft-ietf-lamps-dilithium-certificates-12, IETF (2025). <https://datatracker.ietf.org/doc/draft-ietf-lamps-dilithium-certificates/12/>
 28. Mehmood, A.: Enhancing Network Security: Post-Quantum Cryptography Through Loadable Modules in OpenSSL: An Approach to Enhance OpenSSL’s Cryptographic Agility. Master’s thesis, Tampere University (2024). <https://urn.fi/URN:NBN:fi:tuni-2024122811700>.
 29. Melchor, C.A., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.-C., Gaborit, P., Persichetti, E., Zémor, G.: Hamming Quasi-Cyclic (HQC): Fourth round version. Specification, (2025). https://pqc-hqc.org/doc/hqc-specification_2025-02-19.pdf
 30. Moriarty, K., Kaliski, B., Jonsson, J., Rusch, A.: PKCS #1: RSA Cryptography Specifications Version 2.2. RFC 8017, pp. 1–78. RFC Editor (2016). <https://doi.org/10.17487/RFC8017>. <https://datatracker.ietf.org/doc/rfc8017/>
 31. Mosca, M., Piani, M.: Quantum threat timeline report 2024. Tech. rep., Global Risk Institute (2024). <https://globalriskinstitute.org/publication/2024-quantum-threat-timeline-report/>

32. NIST, Announcing Request for Nominations for Public-Key Post-Quantum Cryptographic Algorithms, (2016). <https://csrc.nist.gov/news/2016/public-key-post-quantum-cryptographic-algorithms>
33. NIST, FIPS 140-2, (2001). <https://csrc.nist.gov/pubs/fips/140-2/upd2/final>
34. NIST, Module-Lattice-Based Digital Signature Standard, (2024). <https://doi.org/10.6028/NIST.FIPS.204>
35. NIST, Module-Lattice-Based Key-Encapsulation Mechanism Standard, (2024). <https://doi.org/10.6028/NIST.FIPS.203>
36. NIST, NIST Releases First 3 Finalized Post-Quantum Encryption Standards, (2024). <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>
37. NIST, NIST to Standardize Encryption Algorithms That Can Resist Attack by Quantum Computers, (2023). <https://www.nist.gov/news-events/news/2023/08/nist-standardize-encryption-algorithms-can-resist-attack-quantum-computers>
38. NIST, NISTIR 8105: Report on Post-Quantum Cryptography, (2016). <https://csrc.nist.gov/News/2016/NIST-Released-NISTIR-8105-Report-on-Post-Quantum>
39. NIST, Stateless Hash-Based Digital Signature Standard, (2024). <https://doi.org/10.6028/NIST.FIPS.205>
40. NIST Computer Security Resource Center, PQC - API Notes, (2017). <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/archive/api-march2017.pdf>
41. NIST PQC Crypto, Post-Quantum Cryptography: Additional Digital Signature Schemes, (2022). <https://csrc.nist.gov/projects/pqc-dig-sig>
42. NIST PQC Crypto, Post-Quantum Cryptography Standardization, (2017). <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization>
43. Open Quantum Safe Project, liboqs, <https://openquantumsafe.org/liboqs/>
44. Open Quantum Safe Project, OQS OpenSSL Provider, <https://openquantumsafe.org/applications/tls.html#oqs-openssl-provider>
45. OpenSSL Management Committee (OMC), OpenSSL 3.0.0 Design, (2020). <https://web.archive.org/web/20240528210436/https://www.openssl.org/docs/OpenSSL300Design.html> (visited on 02/05/2025)
46. Ounsworth, M., Gray, J., Pala, M., Klaußner, J., Fluhrer, S.: Composite ML-DSA for use in X.509 Public Key Infrastructure. I-D draft-ietf-lamps-pq-composite-sigs-07, IETF (2025). <https://datatracker.ietf.org/doc/draft-ietf-lamps-pq-composite-sigs/07/>
47. Popov, A.: Prohibiting RC4 Cipher Suites. RFC 7465, IETF RFC Editor (2015). <https://doi.org/10.17487/RFC7465>
48. PQClean Developers, PQClean, <https://github.com/PQClean/PQClean>
49. Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters. NIST SP 800-186, NIST (2023). <https://doi.org/10.6028/NIST.SP.800-186>
50. Reddy, K. T., Hollebeek, T., Gray, J., Fluhrer, S.: Use of Composite ML-DSA in TLS 1.3. I-D draft-reddy-tls-composite-mldsa-05, IETF (2025). <https://datatracker.ietf.org/doc/draft-reddy-tls-composite-mldsa/05/>
51. Reddy, K. T., Hollebeek, T., Gray, J., Fluhrer, S.: Use of SLH-DSA in TLS 1.3. I-D draft-reddy-tls-slhdsa-01, IETF (2025). <https://datatracker.ietf.org/doc/draft-reddy-tls-slhdsa/01/>

52. Rescorla, E.: The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, pp. 1–160. RFC Editor (2018). <https://doi.org/10.17487/RFC8446>. <https://datatracker.ietf.org/doc/rfc8446/>
53. Rivest, R.L.: The MD5 Message-Digest Algorithm. RFC 1321, IETF RFC Editor (1992). <https://doi.org/10.17487/RFC1321>
54. RustCrypto Developers, RustCrypto: SLH-DSA v0.0.3, (2025). <https://crates.io/crates/slh-dsa/0.0.3>
55. Salzman, P.J., Burian, M., Pomerantz, O.: The Linux Kernel Module Programming Guide, (2007). <https://tldp.org/LDP/lkmpg/2.6/html/lkmpg.html> (visited on 02/02/2025)
56. Schaad, J., Ramsdell, B.C., Turner, S.: Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 4.0 Message Specification. RFC 8551, IETF RFC Editor (2019). <https://doi.org/10.17487/RFC8551>
57. Status Report on the First Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process. Internal Report, NIST (2024). <https://doi.org/10.6028/NIST.IR.8528>
58. Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process. Internal Report, NIST (2025). <https://doi.org/10.6028/NIST.IR.8545>
59. Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process. Internal Report, NIST (2022). <https://doi.org/10.6028/NIST.IR.8413-upd1>
60. Stebila, D., Fluhrer, S., Gueron, S.: Hybrid key exchange in TLS 1.3. I-D draft-ietf-tls-hybrid-design-14, IETF (2025). <https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/14/>
61. Stebila, D., Mosca, M.: Post-quantum Key Exchange for the Internet and the Open Quantum Safe Project. In: Avanzi, R., Heys, H.M. (eds.) SAC. LNCS, vol. 10532, pp. 14–37. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-319-69453-5_2. <https://openquantumsafe.org/>
62. The OpenSSL Project Authors, EVP - high-level cryptographic functions, <https://docs.openssl.org/3.2/man7/evp/> (visited on 02/04/2025)
63. The OpenSSL Project Authors, OpenSSL Provider, <https://docs.openssl.org/3.2/man7/provider/> (visited on 01/30/2025)
64. The OpenSSL Project Authors, openssl-guide-migration, <https://docs.openssl.org/3.2/man7/openssl-guide-migration/#deprecated-function-mappings> (visited on 02/04/2025)
65. Tuveri, N., Brumley, B.B.: Start Your ENGINEs: Dynamically Loadable Contemporary Crypto. In: SecDev, pp. 4–19. IEEE (2019). <https://doi.org/10.1109/SecDev.2019.00014>. <https://doi.org/10.1109/SecDev.2019.00014>
66. Westerbaan, B., Stebila, D.: X25519Kyber768Draft00 hybrid post-quantum key agreement. I-D draft-tls-westerbaan-xyber768d00-03, IETF (2023). <https://datatracker.ietf.org/doc/draft-tls-westerbaan-xyber768d00/03/>